Lab 3: 版本控制系统 & Git

- 什么是版本控制系统
- 本地 Git 仓库
- 远程 Git 仓库
- Github

什么是版本控制系统

- 什么是“版本控制”？我为什么要关心它呢？ 版本控制是一种记录一个或若干文件内容变化，以便将来查阅特定版本修订情况的系统。
- 如果你是位图形或网页设计师，可能会需要保存某一幅图片或页面布局文件的所有修订版本，采用版本控制系统（VCS）是个明智的选择。 有了它你就可以将选定的文件回溯到之前的状态，甚至将整个项目都回退到过去某个时间点的状态，你可以比较文件的变化细节，查出最后是谁修改了哪个地方，从而找出导致怪异问题出现的原因，又是谁在何时报告了某个功能缺陷等等。 使用版本控制系统通常还意味着，就算你乱来一气把整个项目中的文件改的改删的删，你也照样可以轻松恢复到原先的样子。 但额外增加的工作量却微乎其微。
- Git 是一种分布式的版本控制系统。可以用来管理我们的项目代码文件（或其他任何你想管理的项目文件），方便我们对其进行版本控制。

本地 Git 仓库

安装和配置 Git

- 在 Windows 上安装 Git 也有几种安装方法。 官方版本可以在 Git 官方网站下载。 打开 https://git-scm.com/download/win，下载会自动开始。
- 安装完 Git 之后，要做的第一件事就是设置你的用户名和邮件地址。 这一点很重要，因为每一个 Git 提交都会使用这些信息，它们会写入到你的每一次提交中，不可更改：

```
$ git config --global user.name "John Doe"
$ git config --global user.email johndoe@example.com
```

本地 Git 仓库

获取 Git 仓库

- 通常有两种获取 Git 仓库的方式：

  1. 将尚未进行版本控制的本地目录转换为 Git 仓库；
  2. 从其它服务器 克隆 一个已存在的 Git 仓库。

  两种方式都会在你的本地机器上得到一个工作就绪的 Git 仓库。

- 在已存在的目录中初始化仓库

  1. 在命令行中进入该项目目录中
  2. 然后执行 **git init**

```
cd path/of/your/project
git init
```

- 克隆现有的仓库

  可以使用 **git clone <url>** 命令来克隆一个已经存在了的仓库的拷贝。

```
git clone https://github.com/example/example
```

本地 Git 仓库

Git 基础操作

1. Git 会跟踪所有你想要跟踪的文件
2. 在工作目录之下，所有的文件都分为 **已跟踪** 和 **未跟踪** 两种状态
3. 已跟踪 是指那些已经被纳入了版本控制的文件（也就是 Git 已经知道的文件）
4. 工作目录中除已跟踪文件外的其它所有文件都属于未跟踪文件
5. 可以使用 **git status** 命令来查看那些文件处于什么状态

跟踪新文件 / 暂存区

1. 使用 **git add** 命令开始跟踪一个文件，例如：
```
git add README.md
```
2. README.md文件中**此时此刻**的内容就会被 Git 记录到 **暂存区** 中去。
3. 当你继续对该文件进行修改，文件的内容就会和暂存区不一致（可以运行 git status 查看文件的状态），这时你可以再次运行 git add 命令将该文件最新的修改放到暂存区。

本地 Git 仓库

Git 基础操作

提交更新

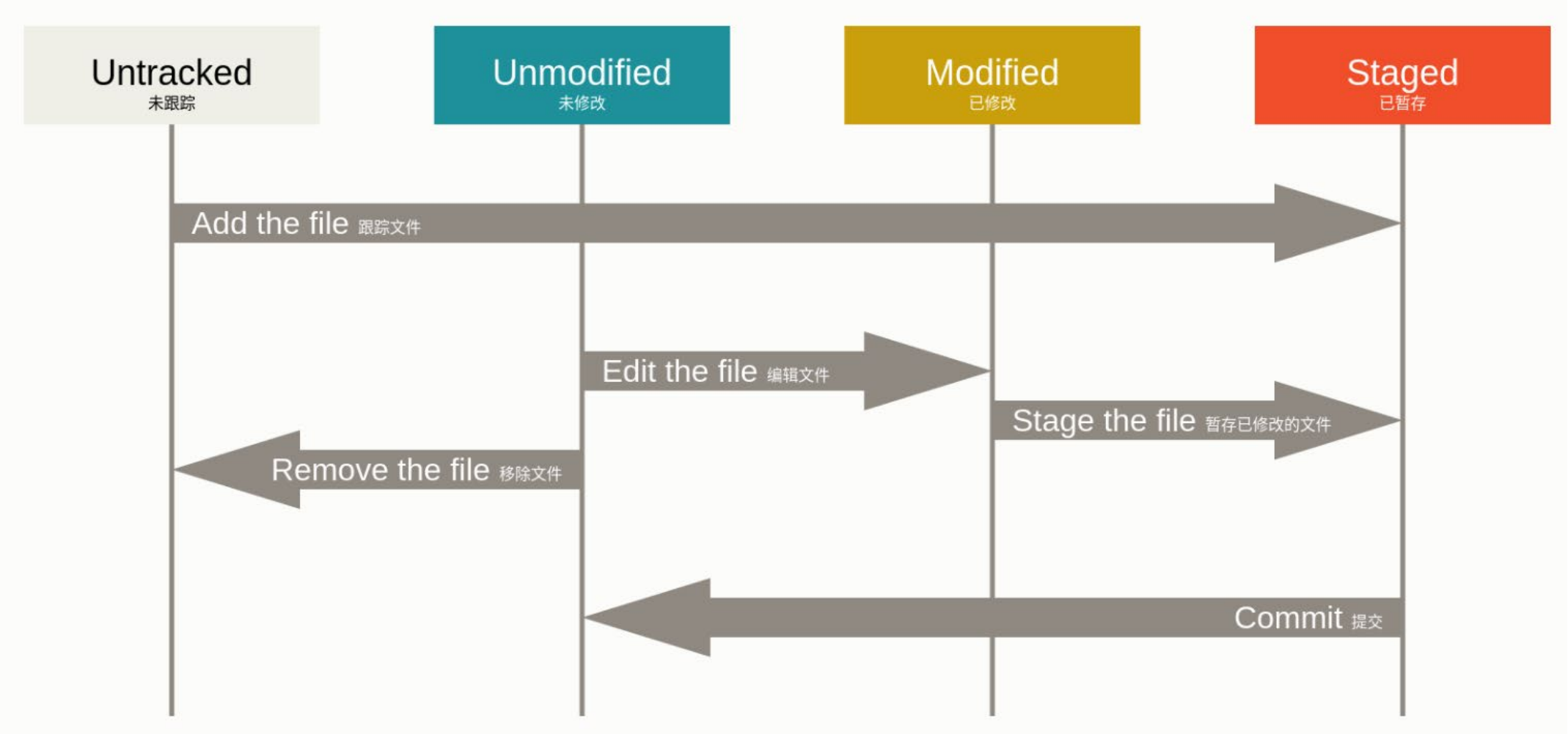
1. 当你已经将所有想要放入一次提交的文件都加入暂存区后，就可以进行**提交**。
2. 执行 **git commit** 命令提交所有已经加入暂存区的文件更改。
3. 这样会启动你选择的文本编辑器来输入提交说明，你可以描述这次提交都做了那些事情。
4. 提交完成后，你可以通过执行 **git log** 命令来查看历史提交
5. 可以看到代表该提交的**SHA-1 校验和**、提交的作者和邮箱、日期、还有提交信息“new commit”

```
C:\Users\lenovo\Desktop\git-test>git log
commit be8594686ae543e7bcaee897a613331355f46292 (HEAD -> master)
Author: LuoMoer <shiweiluomo@gmail.com>
Date:   Sun Mar 15 16:09:31 2026 +0800

    new commit
```

本地 Git 仓库

理解 Git 文件状态



本地 Git 仓库

理解 Git 提交

Git 提交

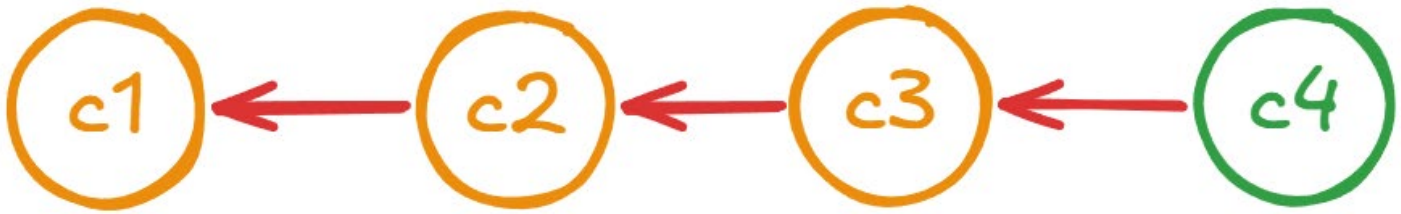
1. 如果不涉及 Git 分支，你在git仓库中所有的提交会串成一个链式结构
2. 其中每一个提交都是该项目在某一时刻的**历史版本**
3. 每当进行一次新提交，其就会指向上一次提交
4. 你可以执行 **git log –graph** 来查看这个结构

本地 Git 仓库

理解 Git 提交

Git 提交

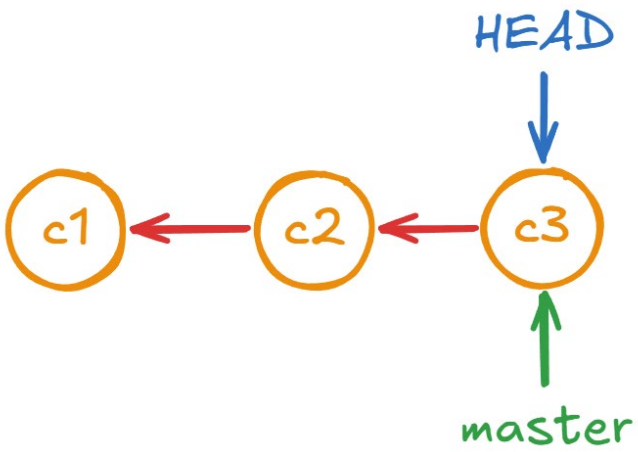
1. 一般来讲，Git 提交只可以创建，不可以删除
2. 也就是说，一旦进行了一次提交，就算你想撤销之前的一些改动，也只能在一个新的提交上回滚改动
3. 如下图所示，为了撤销c3中的某些错误，你需要创建提交c4



本地 Git 仓库

Git 分支

1. Git 中有一种指向提交的指针叫做**分支**
2. Git 的默认分支名字是 master（或main），第一次提交后他就会指向第一个提交。
3. 分支会在每次提交时自动向前移动，经过几次提交之后，master分支应该指向最后的提交。



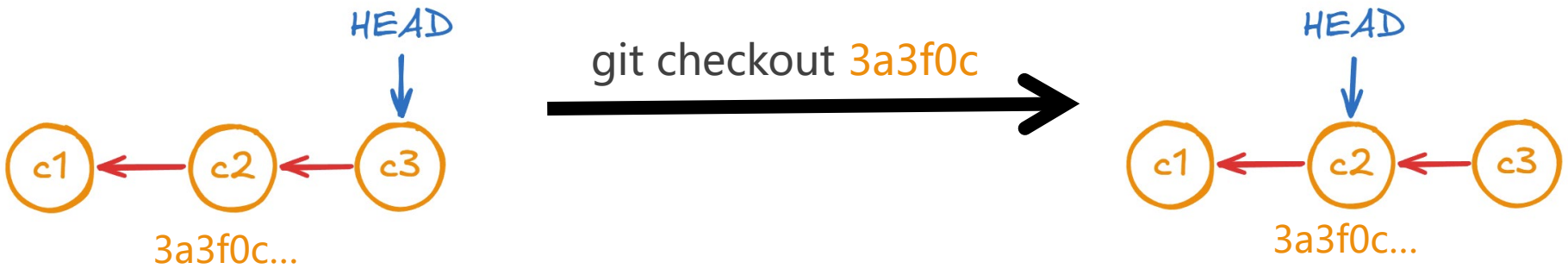
4. HEAD是另一种指针，他的作用主要是指示我们现在处于哪一个分支。

本地 Git 仓库

理解 Git 提交

HEAD 指针

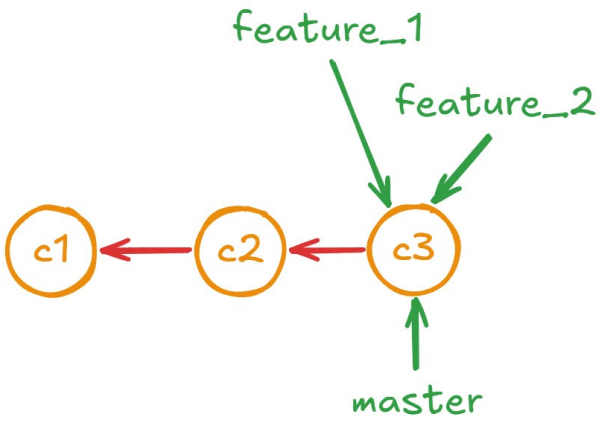
1. HEAD指针代表我们现在在哪一个**分支**，工作目录中的内容将会和HEAD指向的**分支**指向的提交保持一致
2. 我们可以通过移动HEAD来切换当前所处的分支
3. 如果要切换工作目录内容到其他没被分支指向的历史提交上（detach），可以执行 **git checkout <commit_ SHA-1>**



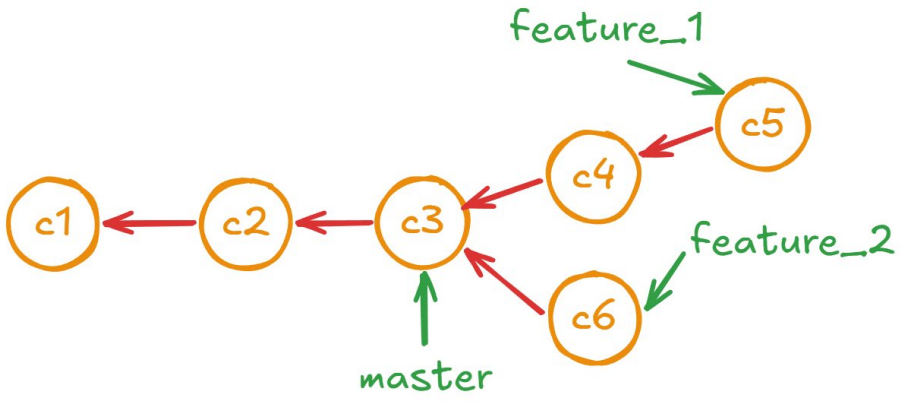
本地 Git 仓库

分支的作用

1. 想象你要同时对你的代码开发两个不同的功能，并且你不希望他们的改动都混到一起去
2. 这时 Git 分支就派上了用场，你可以新建两个分支 feature_1 和 feature_2，然后在这两个分支上分别进行新功能的开发和提交（也许会提交多次），然后依次将他们合并到 master 分支。
3. 在这个过程中，你的提交看起来应该像这样（我们先忽略HEAD指针）：



1. 新建分支

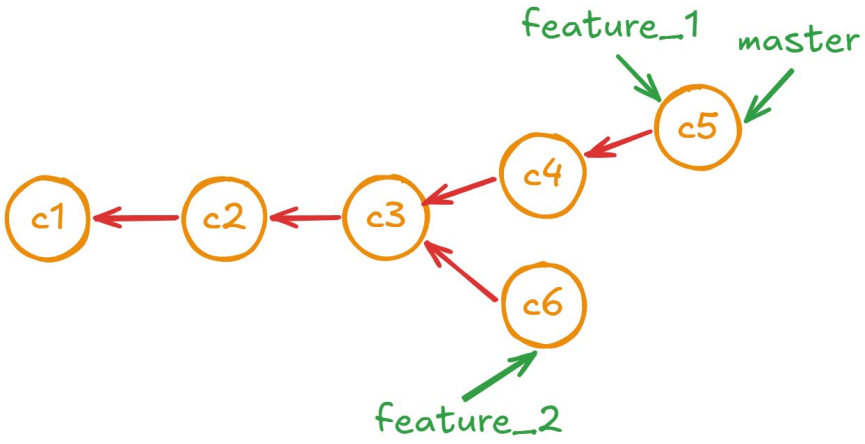


2. 开发新功能

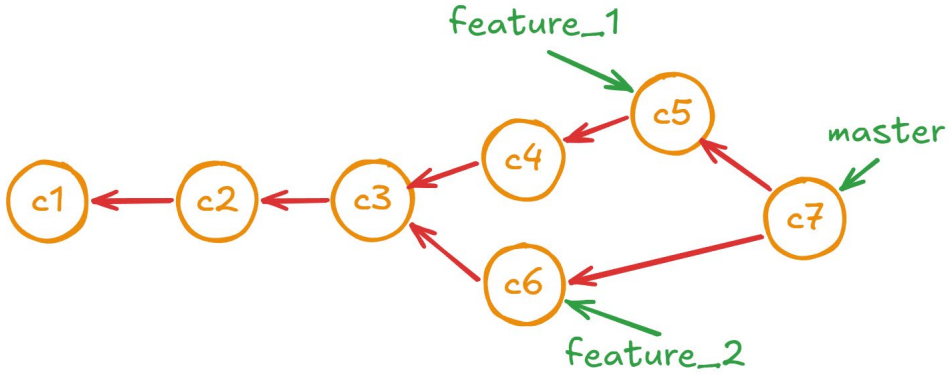
本地 Git 仓库

分支的作用

1. 想象你要同时对你的代码开发两个不同的功能，并且你不希望他们的改动都混到一起去
2. 这时 Git 分支就派上了用场，你可以新建两个分支 feature_1 和 feature_2，然后在这两个分支上分别进行新功能的开发和提交（也许会提交多次），然后依次将他们合并到 master 分支。
3. 在这个过程中，你的提交看起来应该像这样（我们先忽略HEAD指针）：



3. 合并feature_1到master分支



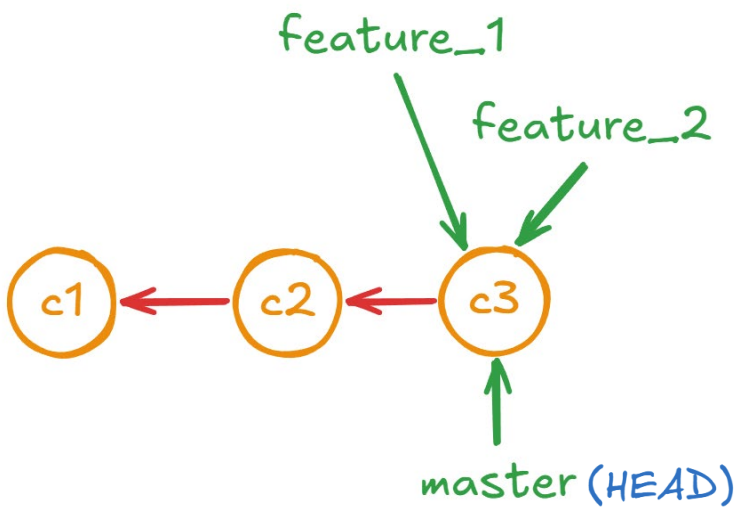
4. 合并feature_2到master分支

本地 Git 仓库

分支操作

- 接下来，我们将通过Git分支操作完成刚刚的过程
- 首先我们需要先**创建分支**，执行 git branch <branch_name> 命令可以在当前提交上创建一个新的分支
- 执行下面的两条命令将创建出两条新的分支，都指向当前的提交：

```
1 git branch feature_1
2 git branch feature_2
```



本地 Git 仓库

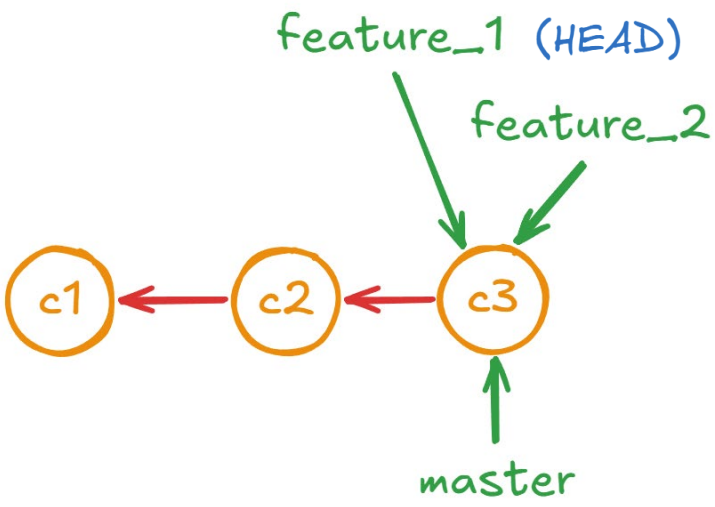
分支操作

- 创建好分支后，如果我们想在某一分支上进行提交，就首先要切换到那个分支
- 我们执行下面的命令切换到feature_1进行开发和提交

```
1 git checkout feature_1
```

- 可以使用下面的命令将创建和切换分支合并

```
1 git checkout -b feature_1
```



本地 Git 仓库

分支操作

- 用同样的方法进行切换分支，我们可以完成feature_1和feature_2的开发
- 最后我们想将这两个功能都合并到 master 分支
- 首先我们合并feature_1分支，先切换到master分支，然后执行：

```
git merge feature_1
```

- 由于C5是C3的直接后继，因此合并分支时Git会直接将master指针向前移动，而不创建新提交，这叫做**快进**

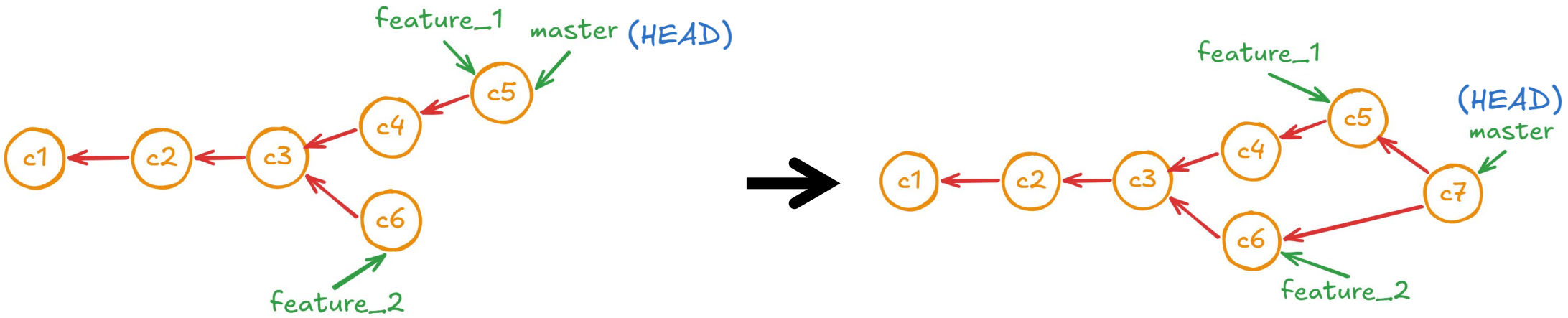
本地 Git 仓库

分支操作

- 我们用同样的方法合并feature 2到master
- 在master分支上，执行：

```
git merge feature_2
```

- 在这种情况下，你的开发历史从一个更早的地方开始分叉开来。 master 分支所在提交并不是 feature_2 分支所在提交的直接祖先，Git 不得不做一些额外的工作。 出现这种情况的时候，Git 会使用两个分支的末端所指的提交（C5 和 C6）以及这两个分支的公共祖先（C3），做一个简单的三方合并。



本地 Git 仓库

分支操作

- 最后，我们已经将 feature_1 和 feature_2 全部合并到 master
- 这两个分支就不再需要了，我们可以执行下面的命令删除他们：

```
1 git branch -d feature_1
2 git branch -d feature_2
```

- 在涉及分支的操作时，你可以在任何时候执行 git branch 来查看自己当前处于那一分支上
- 操作完成后，我们可以执行 git log –graph 看到完整的可视化分支合并过程

```
*   commit 916e5ce95a7057ce8911558eaf1adf8d3dcb88de (HEAD)
|\  Merge: f6b4625 3a3f0cd
| | Author: LuoMoer <shiweiluomo@gmail.com>
| | Date:   Sat Mar 14 20:47:04 2026 +0800
| |
| |     x
| |
| * commit 3a3f0cdaa4da2bdf923edd573dcadf2285116e11 (newbranch)
| | Author: LuoMoer <shiweiluomo@gmail.com>
| | Date:   Sat Mar 14 20:46:04 2026 +0800
| |
| |     x
| |
* | commit f6b4625cda126cf7653354cb8200ae39bd7e24a2
|/  Author: LuoMoer <shiweiluomo@gmail.com>
|   Date:   Sat Mar 14 20:44:47 2026 +0800
|
|       x
|
```

本地 Git 仓库

分支冲突

- 有时候，分支的合并并不像刚刚我们演示的那样顺利
- 如果两个分支对同一份文件中的相同内容做了不同修改，就会引发**分支冲突**
- 这是因为Git不知道应该遵循哪一个分支的内容，需要你手动解决分支冲突，然后告诉Git可以继续合并

远程 Git 仓库

1. 可以通过 **git remote add <remote> <url>** 来用地址添加远程仓库，并为其指定别名
2. 如果是通过 git clone 获取的仓库，那么 git 会自行添加克隆仓库的地址为远程仓库，别名为 **origin**
3. 可以执行 git remote –v 来查看已经配置的远程仓库

```
C:\Users\lenovo\Desktop\gift-app-simple>git remote -v
origin  https://github.com/LuoMoer/gift-app-simple.git (fetch)
origin  https://github.com/LuoMoer/gift-app-simple.git (push)
```

4. 还可以通过 git remote show <remote> 看到更多信息

```
(base) PS C:\Users\lenovo\Desktop\gift-app-simple> git remote show origin
* remote origin
  Fetch URL: https://github.com/LuoMoer/gift-app-simple.git
  Push  URL: https://github.com/LuoMoer/gift-app-simple.git
  HEAD branch: main
  Remote branch:
    main tracked
  Local branch configured for 'git pull':
    main merges with remote main
  Local ref configured for 'git push':
    main pushes to main (up to date)
```

远程 Git 仓库

从远程仓库中抓取与拉取

1. 可以执行 **git fetch <remote>** 访问远程仓库并拉取你本地没有的数据（该操作只会拉取数据，如果你想要合并远程仓库的master分支到本地master分支，则需要在本地master分支手动操作 **git merge <remote>/master** ）
2. 执行 **git pull** 则会尝试抓取数据并自动尝试合并到当前所在分支。

远程 Git 仓库

推送到远程仓库

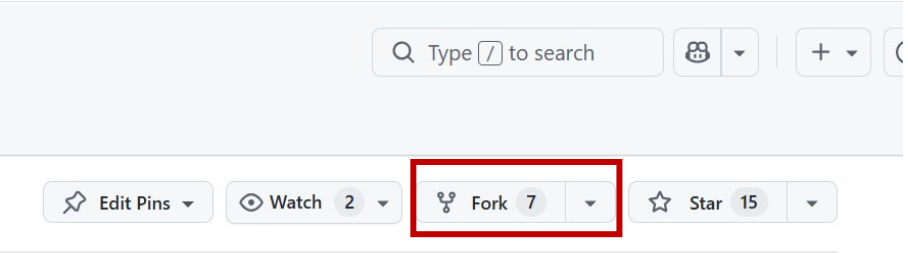
1. 执行 **git push <remote> <branch>** 来将本地仓库的<branch>分支推送到远程仓库<remote>，只有当你有所克隆服务器的写入权限，并且之前没有人推送过时，这条命令才能生效。
2. 当你和其他人在同一时间克隆，他们先推送到上游然后你再推送到上游，你的推送就会毫无疑问地被拒绝。 你必须先抓取他们的工作并将其合并进你的工作后才能推送。

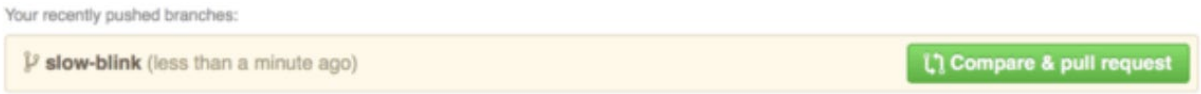
Github

- GitHub 是最大的 Git 版本库托管商，是成千上万的开发者和项目能够合作进行的中心。
- 大部分 Git 版本库都托管在 GitHub，很多开源项目使用 GitHub 实现 Git 托管、问题追踪、代码审查以及其它事情。所以，尽管这不是 Git 开源项目的直接部分，但如果想要专业地使用 Git，你将不可避免地与 GitHub 打交道。

账户的创建和配置

- 要使用Github，你首先需要创建一个免费账户，配置你的邮箱地址。



对项目做出贡献

- 如果你想参与某个项目，但是没有该项目的推送权限，这时就可以对项目进行“派生（Fork）”。 当你“派生”一个项目时，GitHub 会在你的空间中创建一个完全属于你的项目副本，且你对其具有推送权限。
- 这时你就可以在派生的项目上新建分支，然后将新分支推送到 Github。这时你可以在Github上创建一个“拉取请求”（Pull Request，简称PR）供仓库管理者审阅并决定是否合并你的改动。